
Estructuras de Datos

2026
Primer Cuatrimestre

REPASO

TDA Grafo no Dirigido

```
public interface Graph<V, E> {  
    public Iterable<Vertex<V>> vertices();  
    public Iterable<Edge<E>> edges();  
    public Iterable<Edge<E>> incidentEdges(Vertex<V> v);  
    public Vertex<V> opposite(Vertex<V> v, Edge<E> e);  
    public Vertex<V> [] endVertices(Edge<E> e);  
    public boolean areAdjacent(Vertex<V> v, Vertex<V> w);  
    public V replace( Vertex<V> v, V x);  
    public E replace( Edge<E> e, E x);  
    public Vertex<V> insertVertex(V x);  
    public Edge<E> insertEdge(Vertex<V> v, Vertex<V> w, E x);  
    public V removeVertex(Vertex<V> v);  
    public E removeEdge(Edge<E> e); }
```

En Profundidad (DFS)

Algoritmo DFSShell(G : Grafo)

para cada vértice v de G hacer

marcar v como no visitado

para cada vértice v de G hacer

si v no está visitado entonces

DFS(G, v)

$$T(n,m)=O(n+m)$$



En Profundidad (DFS)

Algoritmo DFS(G : Grafo; v : Vértice)

 procesamiento de v previo al recorrido (visita pre)

 marcar a v como visitado

 para cada vértice w adyacente a v en G hacer

 si w no está visitado entonces DFS(G , w)

 procesamiento de v posterior al recorrido (visita pos)

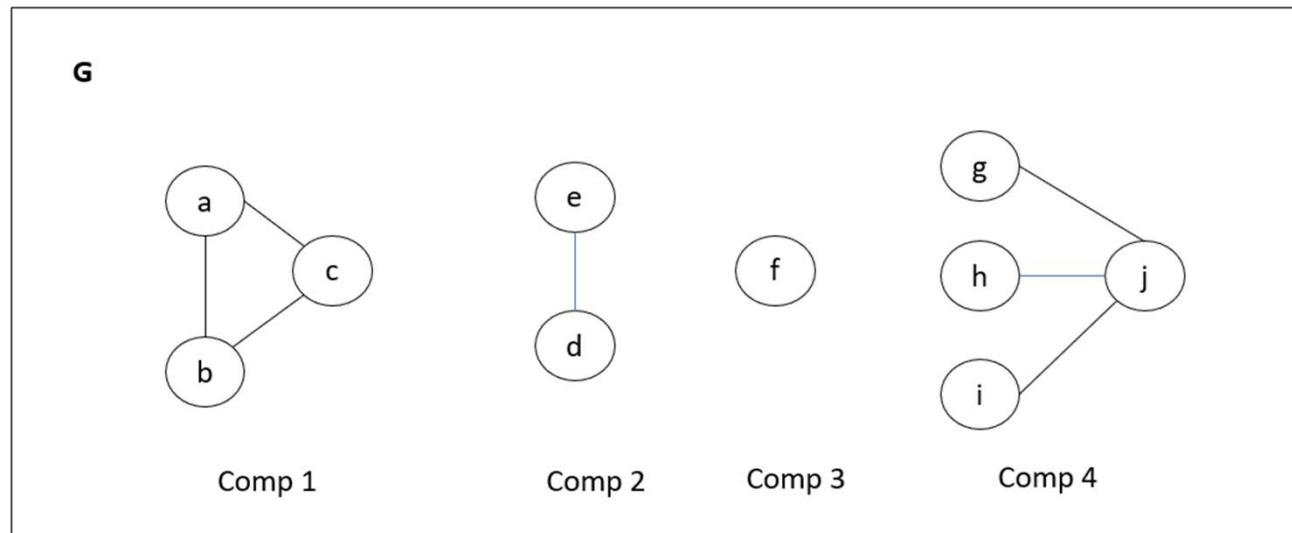
PROBLEMAS

Problema 1

Implemente en Java un método tal que dado un grafo no dirigido genérico G retorne un mapeo con sus componentes conexas numeradas y los vértices que pertenecen a esa componente.



Ejemplo



$M = \{(1, [a, b, c]), (2, [e, d]), (3, [f]), (4, [g, h, i, j])\}$

Algoritmo HallarComponentesConexas (G : Grafo) : Mapeo<Integer, Iterable<Vertex>>

para cada vértice v de G hacer

 marcar v como no visitado

Mapeo <...> resultado = new Mapeo<...>;

int nroComponenteConexa = 0;

para cada vértice v de G hacer

 si v no está visitado entonces

 nroComponenteConexa++;

 verticesComponenteConexa = new ListaDoblementeEnlazada<Vertex>();

 HallarVerticesComponenteConexaActualConDFS(G, v, verticesComponenteConexa);

 resultado.put(nroComponenteConexa, lista);

return resultado;

Algoritmo HallarVerticesComponenteConexaActualConDFS(G : Grafo; v : Vértice;
PositionList<Vertex> lista)

lista.addLast(v);

marcar a v como visitado;

para cada vértice w adyacente a v en G hacer

si w no está visitado entonces

HallarVerticesComponenteConexaActualConDFS(G, w, lista)



Problema 2

Implemente en Java un método tal que dado un grafo no dirigido genérico G y dos vértices, retorne un camino entre ambos vértices.

Si no existe un camino entre los vértices dados el método debe retornar nulo.

Algoritmo CaminoShell(G : Grafo; origen, destino: Vertex) : PositionList<Vertex>

para cada vértice v de G hacer

 marcar v como no visitado

PositionList<Vertex> resultado = new ListaDoblementeEnlazada<Vertex>()

encontreCamino = CaminoDFS(G, origen, destino, resultado)

si encontreCamino entonces return resultado else return null

Algoritmo CaminoDFS(G : Grafo; origen, destino : Vértice; PositionList<Vertex<V>> lista) : boolean

 marcar a origen como visitado;

 lista.addLast(origen);

 if (origen == destino) { return true; }

 else

 encontre = false;

 para cada vértice w adyacente a origen en G AND !encontre hacer

 si w no está visitado entonces

 encontre = CaminoDFS(G, w, destino, lista)

 si encontre { return true; } sino {lista.remove(lista.last()); return false; }

Problema 3

Implemente en Java un método tal que dado un grafo no dirigido genérico G y dos vértices, retorne un listado de todos los caminos entre ambos vértices.

Si no existe un camino entre los vértices dados el método debe retornar una lista nula.

```
Algoritmo CaminosShell( G : Graph<V,E>; origen, destino: Vertex<V> ) :  
    PositionList<PositionList<Vertex<V>>>
```

```
    para cada vértice v de G hacer
```

```
        marcar v como no visitado
```

```
    PositionList<Vertex> camino = new Lista<Vertex<V>>();
```

```
    PositionList<PositionList<Vertex<V>>> todos =
```

```
        new Lista<PositionList<Vertex<V>>>();
```

```
    CaminosDFS( G, origen, destino, resultado, todos, camino );
```

```
    return todos;
```

Algoritmo CaminosDFS($G : \text{Graph}\langle V, E \rangle$; origen, destino : Vértice; $\text{PositionList}\langle \text{PositionList}\langle \text{Vertex}\langle V \rangle \rangle \rangle$ todos;
 $\text{PositionList}\langle \text{Vertex}\langle V \rangle \rangle$ camino)

 marcar a origen como visitado;

 camino.addLast(origen);

 if (origen == destino) { todos.addLast(clonar(camino)) }

 else

 para cada vértice w adyacente a origen en G hacer

 si w no está visitado entonces

 CaminosDFS($G, w, \text{destino}, \text{todos}, \text{camino}$)

 camino.remove(camino.last())

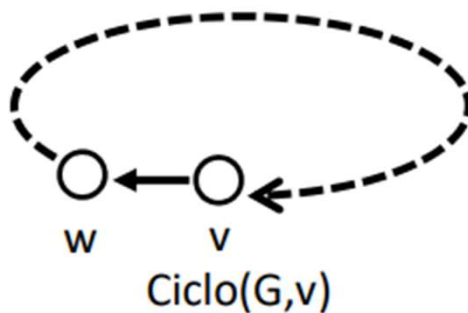
 marcar a origen como NO visitado

Problema 4

El problema de encontrar un camino es útil a la hora de determinar si en un grafo G existe un ciclo que contenga a un vértice v .

La estrategia consiste en buscar caminos desde los adyacentes de v (que se llaman w) hacia v .

HallarCamino(G, w, v, camino)



Algoritmo HallarCiclo(G, v) : Lista

 encontre $\leftarrow$ falso

 mientras hay adyacentes para considerar y no encontre hacer

 w $\leftarrow$ siguiente adyacente de v en G

 encontre $\leftarrow$ HallarCamino(G, w, v, camino)

 finmientras

 ciclo $\leftarrow$ new Lista()

 si encontre entonces

 ciclo.addLast(v)

 para cada vertice x de camino hacer

 ciclo.addLast(x)

 finpara

 finsi

 retornar ciclo

$$T_{\text{hallarciclo}}(n,m) = O(\text{grado}(v) * (n+m) + n)$$

Break.

Problema 5

Dado un grafo G con vértices rotulados con caracteres y arcos rotulados con números enteros y dos vértices origen y destino, escriba un método que permita hallar el camino de costo mínimo entre los vértices dados.

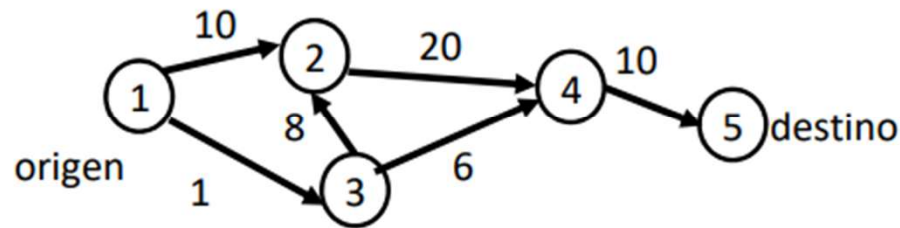


La estrategia consiste de generar por fuerza bruta todos los caminos posibles de origen a destino con su costo.

Hago un proceso de mínimo: Cada vez que encuentro un nuevo camino de origen a destino, me fijo si el costo del nuevo camino hallado es mejor que el mejor que tengo hasta este momento. Si es así, lo actualizo.

Para generar todos los caminos posibles debo:

- (1) Marcar vértices como visitados para no entrar en bucles infinitos si el grafo tiene ciclos dirigidos.
- (2) Desmarcar vértices al hacer backtracking para permitir que un vértice pueda pertenecer a más de un camino posible.



*El camino 1-2-4-5 tiene costo
 $10+20+10=40$*

*El camino 1-3-2-4-5 tiene costo
 $1+8+20+10=39$*

*El camino 1-3-4-5 tienen costo
 $1+6+10=17$*

*Para explorar el segundo y
tercer caminos es necesario
desmarcar los vértices ya
visitados al generar el primer
camino.*

Algoritmo HallarCaminoMinimo(G, origen, destino, camino_actual,
costo_camino_actual, camino_minimo, costo_camino_minimo)
origen.put(Estado, Visitado) { marco a origen como visitado }
Camino_actual.addLast(origen) { encolo a origen en el camino_actual }
Si origen = destino **entonces** { llegué de nuevo a destino }
 Si costo_camino_actual < costo_camino_minimo **entonces**
 { encontré un camino mejor => actualizo el mejor camino hasta el momento }
 camino_minimo ← camino_actual.clone_profundidad()
 costo_camino_minimo ← costo_camino_actual
Sino { origen <> destino => para cada adyacente de origen, veo si puedo llegar a destino }
 para cada adyacente v de origen en G **hacer**
 si v.get(Estado) = NoVisitado **entonces**
 HallarCaminoMinimo(G, v, destino, camino_actual,
 costo_camino_actual + peso_arco(origen,v), camino_minimo,
 costo_camino_minimo)
camino_actual.remove(camino_actual.last()) { *backtracking: saco a origen del camino* }
origen.put(Estado, NoVisitado) { desmarco a origen así se puede reusar para otros caminos }

Algoritmo CaminoMínimoShell(G : Grafo; origen, destino: Vertex) :
PositionList<Vertex>

INFINITO = Float.MAX_VALUE

para cada vértice v de G hacer

 marcar v como no visitado

Camino resultado = new Camino()

Camino actual = new Camino()

actual.setCosto(*INFINITO*)

CaminoMinimoDFS(G, origen, destino, resultado, actual);

return resultado.getVertices();

Camino<V>

-vertices :

PositionList<Vertex<V>>

-costo: float

```
public class Camino<V> {  
    protected PositionList<Vertex<V>> vertices;  
    protected float costo;  
    public Camino() {  
        this.camino = new ListaDobleEnlazada<Vertex<V>>();  
        this.costo = 0.0f; }  
    public PositionList<Vertex> getVertices()  
    public void setVertices(PositionList<Vertex<V>> nuevo)  
    public void setCosto (int costo) { this.costo=costo; }  
    public int getCosto( ) {return this.costo;}  
}
```

Algoritmo CaminoMinimoDFS(G : Grafo; origen, destino : Vértice; resultado, actual : Camino)

 marcar a origen como visitado;

 actual.getCamino().addLast(origen);

 if (origen == destino) {

 if(resultado.getCosto() < actual.getCosto()) {

 resultado.setCamino(clonar (actual.getCamino())); resultado.setCosto(actual.getCosto()); }

 else

 para cada vértice w adyacente a origen en G a través del arco e hacer

 si w no está visitado entonces

 actual.setCosto(actual.getCosto() + e.element());

 CaminosDFS(G, w, destino, todos, camino)

 actual.setCosto (actual.getCosto() - e.element());

 camino.remove(camino.last())

 marcar a origen como NO visitado

Para pensar...

¿Qué cambiaría en estos ejercicios en el caso de que el grafo G sea dirigido?



THE END.